You said:



HOW TO RUN THIS CODE IN VSC

ChatGPT said:

Bro, your code is almost ready! 🔥 From your screenshot, you're using VS Code on Windows, and your project imports `pygame` and a custom module called `game.game_engine`.

Follow these steps to run it.

Step 1: Open the VS Code terminal

You already have the terminal open at the bottom. Make sure you're in the folder containing `main.py`.

Run:

```
cd C:\Users\lbhim\Downloads
```

Step 2: Check Python installation

Run these commands one by one:

```
python --version
```

Then:

```
python -m pip --version
```

If both commands work, continue to Step 3.

Step 3: Install Pygame

Run:

```
python -m pip install pygame
```

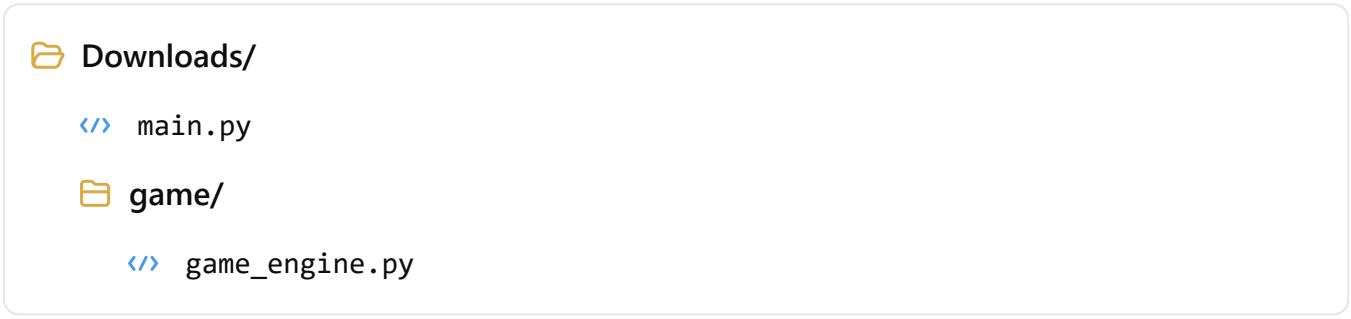
Wait until the installation finishes.

Step 4: Check your project files

Your code contains this import:

```
from game.game_engine import GameEngine
```

So your project should have this structure:



If the `game` folder or `game_engine.py` file is missing, the program won't run. Make sure the files from your project are in the correct locations.

Step 5: Run your game 🎮

In the terminal, execute:

```
python main.py
```

A Pygame window should open with your Number Guessing Game.

Step 6: If you get an error

Here are the common errors and fixes:

Error	Fix
`No module named 'pygame'`	Run `python -m pip install pygame`
`No module named 'game'`	Open the correct project folder
`No module named 'game.game_engine'`	Check whether `game_engine.py` exists inside `game`
`python is not recognized`	Install Python and enable **Add Python to PATH**

Window opens and closes immediately	Run it from the terminal using `python main.py` to see the error

One important thing: In your screenshot, the terminal shows a command that looks like `C:\Users\lbhim\Downloads\main.py`, followed by another prompt. That isn't the normal command for running a Python script. Use `python main.py` instead.

If it still doesn't work, **copy and send me the complete error shown in the terminal**, and I'll tell you exactly what to do next.

You said:



IM GETTING ERROR IN TIS

ChatGPT said:

Bro, I found the problem! Your error says:

```
python is not recognized as the name of a cmdlet
```

This means **Python is either not installed on your Windows laptop, or it is installed but not added to PATH.** Your game code isn't the problem yet. Let's fix Python first.

Step 1: Check whether Python is already installed

In the VS Code terminal, type:

```
py --version
```

Press Enter.

- If it shows something like `Python 3.13.x`, Python is installed. Skip to Step 3.
- If `py` is also not recognized, continue to Step 2.

Step 2: Install Python if needed

1. Open the official download page: Download Python for Windows ↗.
2. Download and install Python using the instructions on the page. Python 3.14.... +1
3. If you use the traditional Windows installer, tick **Add Python to PATH** before clicking Install.

4. After installation, **close VS Code completely and reopen it**.

Step 3: Verify the installation

Open a new VS Code terminal and run:

```
python --version
```

Then install Pygame:

```
python -m pip install pygame
```

Step 4: Run your game

```
cd C:\Users\lbhim\Downloads
python main.py
```

Do this first, bro: Run `py --version` and tell me what appears. That will help us avoid reinstalling Python unnecessarily.

You said:



IM GETTING ERROR CORRECT IT

ChatGPT said:

Bro, good news! 🔥 Python and Pygame are installed correctly now. The current error is:

```
ModuleNotFoundError: No module named 'game'
```

This means Python cannot find your `game` **folder or the** `game_engine.py` **file**. Let's fix it.

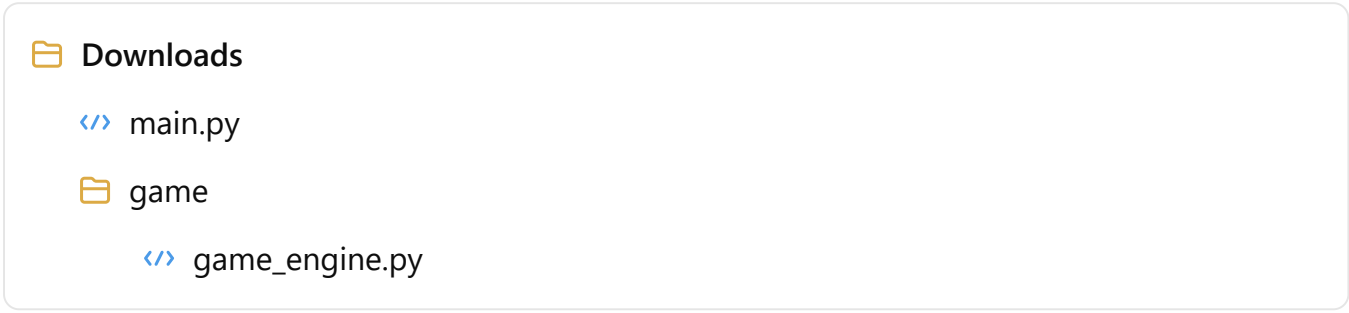
Step 1: Check your project files

In the VS Code terminal, run:

```
dir
```

Look for `main.py` and a folder named `game`.

Your project should look like this:



Step 2: Find the missing file

Run this command:

```
Get-ChildItem "C:\Users\lbhim\Downloads" -Recurse -Filter game_engine.py
```

- **If it finds** `game_engine.py`: note the folder path and send it to me.
- **If it finds nothing:** you need to download or extract the original project files containing `game_engine.py`.

Step 3: Run the game again

Once the folder structure is corrected, run:

```
py main.py
```

The `pkg_resources` warning in your screenshot is not the main problem; you can ignore it for now.

Do this first: Run `dir` in your terminal and send me the output or a screenshot. I'll tell you exactly which folder or file needs fixing.

You said:



ERROR??

ChatGPT said:

Bro, now it's confirmed! The command ran, but **it did not find** `game_engine.py` **anywhere inside your Downloads folder.** That's why `py main.py` gives this error:

```
ModuleNotFoundError: No module named 'game'
```

Your Pygame installation is working correctly. The missing part is your game's source code.

Fix it step by step

Step 1: Check your Downloads folder

In VS Code, look at the Explorer panel on the left. Do you have a folder named `game` containing `game_engine.py` ?

Step 2: Search your whole computer for the file

Run this in PowerShell:

```
Get-ChildItem C:\Users\lbhim -Recurse -Filter game_engine.py -ErrorAction SilentlyContinue
```

This may take a little time.

Step 3: If nothing appears

You need the original project files. Your `main.py` imports `GameEngine` from another Python file, so `main.py` alone is not enough to run the game.

Send me one thing: Where did you get this Number Guessing Game project?

- Did you download a ZIP file?
- Did you copy only `main.py` ?
- Did you create the code yourself?

Tell me which one, and I'll guide you through the exact fix.

You said:

IT STUCKED WHAT SHD I DO

ChatGPT said:

Bro don't worry 😭❤️ Your terminal is probably stuck because the command is searching your entire user folder. Let's stop it safely.

Step 1: Stop the command

In the VS Code terminal:

1. Click inside the terminal.
2. Press **Ctrl + C** on your keyboard.
3. Wait until you see the prompt again, like this:

```
PS C:\Users\lbhim\Downloads>
```

If it doesn't stop, click the **trash bin icon** 🗑️ in the terminal panel to kill that terminal, then open a new one using **Terminal → New Terminal**.

Step 2: Don't run the big search again

We already know that `main.py` is missing its `game` module. We don't need to search your whole computer right now.

Instead, run this simple command:

```
dir C:\Users\lbhim\Downloads -Directory
```

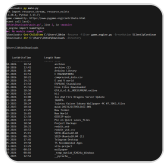
This will list only the folders in Downloads.

Step 3: Send me the result

Take a screenshot of the folder list and send it here. I'll help you locate the project files and get the game running.

Don't close VS Code or delete any files. We can fix this step by step.

You said:



CHECK IT

ChatGPT said:

Bro, I checked your screenshot. 👍 Your terminal is working properly now! The problem is that **there is no `game` folder visible in your Downloads directory listing.** Your `main.py` needs that folder to run.

Let's check whether your project is inside one of the other folders.

Step 1: Check the `Project Backups` folder

Copy and run this command:

```
dir "C:\Users\lbhim\Downloads\Project Backups"
```

Step 2: If you don't find the game files

Check the `archive` folders for a ZIP containing your game project. Don't delete or move anything yet.

Step 3: Tell me how you got `main.py`

Did you download a complete Number Guessing Game project, or did you create/download only `main.py` ?

Important: Don't run `py main.py` again just yet. We need to locate `game_engine.py` first; installing Pygame again will not fix this missing-module error.

You said:



I DOWNLOADED IT FROM GIT HUB HOW TO ACCESS THT FILE

ChatGPT said:

Bro, got it! Since you downloaded the game from GitHub, you need to download or extract the **complete repository**, not just `main.py`. Your `main.py` depends on `game/game_engine.py`, which is currently missing from the folder you're running it in.

Let's get the original files.

Step 1: Open your GitHub project

1. Open the GitHub repository page where you downloaded the game.
2. Click the green **Code** button.
3. Select **Download ZIP**.
4. Wait for the ZIP file to download.

Step 2: Extract the ZIP file

1. Open **File Explorer** → **Downloads**.
2. Find the ZIP file you downloaded from GitHub.
3. Right-click it and choose **Extract All...**.
4. Click **Extract**.

Step 3: Open the complete project in VS Code

In VS Code, click **File** → **Open Folder...** and select the extracted project folder.

Your files should look something like this:

```
Number-Guessing-Game/
    main.py
    game/
        game_engine.py
```

The exact structure may differ depending on the repository.

Step 4: Run the game

Open the VS Code terminal (**Terminal** → **New Terminal**) and run:

```
py -m pip install pygame
py main.py
```

If the repository contains a `requirements.txt` file, install its dependencies with `py -m pip install -r requirements.txt` instead.

If you still can't find the file: send me the **GitHub repository link** you downloaded the game from. I'll help you identify the correct folder and the exact files you need.

You said:

text_box.py
Code

game_engine.py
Code

main.py
Code

OK I FORGOT TO DWONLOAD THIS TWO OTHER FILES SO NOW TELL ME HOE TO RUN THIS CODE IN VSC

ChatGPT said:

Bro 🔥 now you have all 3 files! I'll show you exactly how to arrange them and run your Number Guessing Game in VS Code.

Step 1: Create the correct folder structure

Your code uses imports like `from game.game_engine import GameEngine`, so **the folder names must be correct**.

Open File Explorer and go to `C:\Users\lbhim\Downloads`.

Create this structure:

```
Downloads
    main.py
    game
        game_engine.py
        text_box.py
```

How to arrange the files

1. Keep `main.py` directly inside `Downloads`.
2. Create a folder named `game` inside `Downloads`.
3. Move `game_engine.py` and `text_box.py` into that `game` folder.

4. Make sure the filenames are exactly as shown, not `game_engine.py.txt` .

Step 2: Open the project in VS Code

In VS Code, click **File** → **Open Folder...**

Select:

```
C:\Users\lbhim\Downloads
```

Then click **Select Folder**.

Step 3: Install Pygame

Open **Terminal** → **New Terminal** and run:

```
py -m pip install pygame
```

You've already installed it, so this command will simply confirm that it's available.

Step 4: Run your game

In the terminal, run:

```
cd C:\Users\lbhim\Downloads
py main.py
```

A game window should open with the title **Number Guessing Game – Pygame Edition**.

- Enter a number between 1 and 100.
- Click **SUBMIT** or press Enter.
- Guess higher or lower based on the feedback.
- After winning, press **R** to start a new game.

If you get another error

Send me a screenshot of the VS Code Explorer panel on the left showing `main.py` , the `game` folder, and its two files. I'll help you correct the structure.

You said:



I HAVE ADDED LIKE THIS NOW TELL ME HOW TO RUN THID GAME

ChatGPT said:

Bro 🔥 you're almost there! I can see your three files open in VS Code, but **you haven't opened a project folder yet**, and the files appear to be directly inside `Downloads`. Let's set it up correctly.

Step 1: Create the `game` folder

In VS Code's Explorer on the left:

1. Click **File → Open Folder...**
2. Select `C:\Users\lbhim\Downloads` and click **Select Folder**.
3. In the Explorer, click the **New Folder** icon and name the folder exactly:

```
game
```

4. Move `game_engine.py` and `text_box.py` into the `game` folder using File Explorer.

Your structure must look like this:

```
Downloads
    main.py
    game/
        game_engine.py
        text_box.py
```

Step 2: Run the game

Click **Terminal → New Terminal** in VS Code, then execute these commands one at a time:

```
cd C:\Users\lbhim\Downloads
```

```
py -m pip install pygame
```

```
py main.py
```

Step 3: Play the game 🎮

When the window opens:

- Enter a number between 1 and 100.
- Click **SUBMIT** or press Enter.
- Follow the **TOO LOW** or **TOO HIGH** message.
- After winning, press **R** to play again.

Important: If the game still gives an error, send me a screenshot of the Explorer after creating the `game` folder. We'll fix the exact issue.

You said:

SEE I DONT MADE A FOLDER SO CAN U GIVE ME SPECIFIC STEPS I JUST GOT THIS 3 FIKES

ChatGPT said:

Bro, no problem! ❤️ You have only 3 files, and that's enough. I'll show you how to create the required folder from scratch, step by step. You don't need to download anything else.

Step 1: Create the `game` folder

You currently have these three files in `C:\Users\lbhim\Downloads`:

- `main.py`
- `game_engine.py`
- `text_box.py`

First, **open Windows File Explorer** (press `Windows + E`).

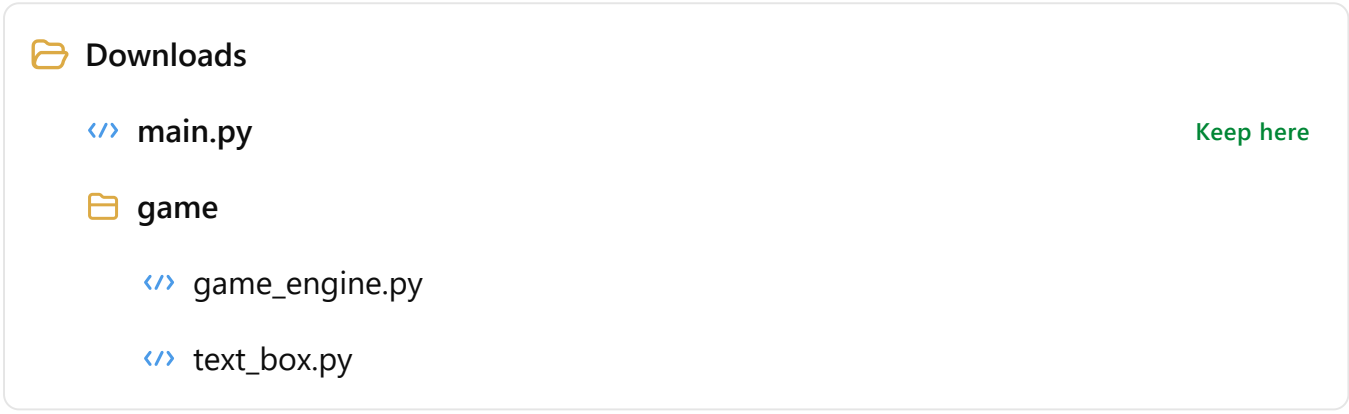
1. Go to **Downloads**.
2. Right-click an empty space.
3. Select **New** → **Folder**.
4. Name the folder exactly `game` and press Enter.

Step 2: Move two files into the folder

Your Downloads folder should now have a new folder named `game`.

1. Select `game_engine.py`.
2. Hold **Ctrl** and select `text_box.py` too.
3. Press **Ctrl** + **X** to cut both files.
4. Double-click the `game` folder.
5. Press **Ctrl** + **V** to paste both files inside.

Your final structure must look exactly like this:



Step 3: Run the game in VS Code

Open VS Code. In the top menu, click **Terminal** → **New Terminal**.

Copy and run these commands one at a time:

Command 1

```
cd C:\Users\lbhim\Downloads
```

Command 2

```
py -m pip install pygame
```

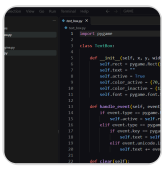
Command 3

```
py main.py
```

That's it! Your Number Guessing Game should open in a separate window. 🎮

Important: Do not move `main.py` into the `game` folder. Only `game_engine.py` and `text_box.py` go inside it.

You said:



SEE ITS LOOKIJNG LIKE THIS BUT WHAT ABT MAIN.PY

ChatGPT said:

Bro, you're doing it correctly! 🔥 I can see your `GAME` folder contains `game_engine.py` and `text_box.py`. Now we just need to put `main.py` in the correct place.

Step 1: Go back to Downloads

1. Press **Windows + E** to open File Explorer.
2. Open `C:\Users\lbhim\Downloads`.
3. Find `main.py` there. It should be **outside** the `GAME` folder.

Your structure should look like this:

- **Downloads**
  - **main.py** Here
  - **GAME**
    - game_engine.py
    - text_box.py

Step 2: Open `main.py` in VS Code

- Click **File → Open File...**
- Navigate to `C:\Users\lbhim\Downloads`.
- Select `main.py` and click **Open**.

If you can't find it in Downloads, tell me and we'll locate it.

Step 3: Run the game

Click **Terminal** → **New Terminal**, then enter:

```
cd C:\Users\lbhim\Downloads
py main.py
```

Important: Your folder is named `GAME` in the screenshot, but your code imports `game.game_engine`. To avoid an import error on Windows, it's best to rename `GAME` to `game` using File Explorer before running the command.

Once `main.py` is in Downloads and the folder is named `game`, run the command above. 🎮

You said:



ITS LOOKING LIKE THIS IS THIS OKAY

ChatGPT said:

YES BRO! 🔥 Your `main.py` is in the correct location, and I can see both `game_engine.py` and `text_box.py` inside your `GAME` folder.

There's just **one small thing left** before running the game.

Step 1: Rename your folder

In the VS Code Explorer on the left:

1. Right-click the `GAME` folder.
2. Select **Rename**.
3. Change `GAME` to `game` (all lowercase).
4. Press Enter.

Your structure should look like this:

```
Downloads
    main.py
    game
```

```
game_engine.py
text_box.py
```

Step 2: Run your game 🎮

Click **Terminal** → **New Terminal** in VS Code.

Enter these commands one at a time:

```
cd C:\Users\lbhim\Downloads
```

```
py main.py
```

Your Number Guessing Game should open in a separate window!

Step 3: Play

- Enter a number between 1 and 100.
- Click **SUBMIT** or press Enter.
- Follow the hints to guess the secret number.
- Press **R** after winning to restart.

If an error appears, send me a screenshot of the terminal. We're very close now! 💪

You said:



hey still getting error and whats problem with the folder name

ChatGPT said:

Bro, I found the actual problem! Don't worry — we can fix this. 🔥

Your error is:

```
ModuleNotFoundError: No module named 'game'
```

The folder name is not the real problem. Python can usually import a folder named `GAME` on Windows. The issue is that your `main.py` is in `Downloads`, while your two other files appear to be inside `Downloads\GAME`. Let's verify the folder and fix the structure.

Step 1: Check your folder structure

In your VS Code terminal, copy and run:

```
dir C:\Users\lbhim\Downloads\GAME
```

You should see both `game_engine.py` and `text_box.py`.

Step 2: Create the correct package folder

Run this command:

```
New-Item -ItemType Directory -Force C:\Users\lbhim\Downloads\game
```

Now copy the two files into it:

```
Copy-Item C:\Users\lbhim\Downloads\GAME\game_engine.py
C:\Users\lbhim\Downloads\game\
Copy-Item C:\Users\lbhim\Downloads\GAME\text_box.py C:\Users\lbhim\Downloads\game\
```

Create an empty `__init__.py` file:

```
New-Item -ItemType File -Force C:\Users\lbhim\Downloads\game\__init__.py
```

Your project will now look like this:

```
Downloads/
├── main.py
├── GAME/
│   ├── game_engine.py
│   └── text_box.py
└── game/
    ├── __init__.py
    ├── game_engine.py
    └── text_box.py
```

Step 3: Run the game

Execute:

```
cd C:\Users\lbhim\Downloads
py main.py
```

This should resolve the missing `game` module error.

If you get a different error, send me the new terminal screenshot and we'll fix that next.

You said:



im getting error

ChatGPT said:

Bro, don't worry! 😄 This error is actually good news — it means the files are **already in the `game` folder**. PowerShell says it cannot copy them because the source and destination are the same files.

You don't need to copy them again. Let's continue.

Step 1: Check your files

Copy this command into your VS Code terminal:

```
dir C:\Users\lbhim\Downloads\game
```

You should see `game_engine.py` and `text_box.py`.

Step 2: Create `__init__.py`

Run this command:

```
New-Item -ItemType File -Force C:\Users\lbhim\Downloads\game\__init__.py
```

This marks `game` as a Python package.

Step 3: Run your game

Run these commands one by one:

```
cd C:\Users\lbhim\Downloads
```

```
py main.py
```

Your game should launch if the files are in the correct places.

If you still get an error, send me the latest terminal screenshot. We'll fix the next issue based on the exact message.

You said:



im still getting error

ChatGPT said:

Bro, I can see the exact problem now! Your `game` folder contains both Python files, and `__init__.py` is created correctly. But Python still cannot find the folder when running `main.py`.

Let's fix this directly. Follow these steps exactly.

Step 1: Check where `main.py` is located

Your terminal shows that `main.py` is here:

```
C:\Users\lbhim\Downloads\main.py
```

That's fine. The issue is that Python isn't finding the `game` package from that location, even though you've created it.

Step 2: Run this diagnostic command

Copy and paste this into your VS Code terminal:

```
py -c "import os, sys; print('Folder:', os.getcwd()); print('Files:', os.listdir('.')); print('Python path:', sys.path[0])"
```

This will show what Python can see in your Downloads folder.

Step 3: Try running the game with this command

```
py -c "import sys; sys.path.insert(0, r'C:\Users\lbhim\Downloads'); import game.game_engine; print('Game module found!')"
```

- If it prints `Game module found!`, try `py main.py` again.
- If it gives another error, send me that output.

One more thing: Your terminal shows the folder `game` has `game_engine.py`, `text_box.py`, and `__init__.py`. You do not need to copy those files again. We'll diagnose why Python isn't finding them before changing anything else.

You said:



hmm

ChatGPT said:

Bro 😭 I see the issue now. The diagnostic command confirms Python still cannot find the `game` package. Let's stop repeating the same commands and check the exact folder location.

Step 1: Run this command

Copy and paste this into your VS Code terminal:

```
Test-Path "C:\Users\lbhim\Downloads\game"
```

It will return either `True` or `False`.

Step 2: Check the folder contents

Then run:

```
Get-ChildItem "C:\Users\lbhim\Downloads\game" -Force
```

You should see:

- `game_engine.py`
- `text_box.py`
- `__init__.py`

Step 3: Send me the result

Tell me whether Step 1 returns `True` or `False`, and send the output of Step 2.

Don't delete, move, or copy any more files yet. We'll use these results to fix the import error directly.

You said:



error!!

ChatGPT said:

Bro, GOOD NEWS! 🎉 Your screenshot confirms that the `game` folder exists and all three required files are inside it. The problem is still the Python import, so let's check something specific.

Step 1: Run this exact command

Copy and paste into your VS Code terminal:

```
py -c "import os; print(os.path.isdir(r'C:\Users\lbhim\Downloads\game')); print(os.listdir(r'C:\Users\lbhim\Downloads\game'))"
```

You should see `True` and a list containing `game_engine.py`, `text_box.py`, and `__init__.py`.

Step 2: Try this import test

```
py -c "import importlib.util; print(importlib.util.find_spec('game'))"
```

If it prints `None`, Python isn't resolving the package as expected.

Step 3: Quick fix to try

Run this command to explicitly set the module search path:

```
$env:PYTHONPATH = "C:\Users\lbhim\Downloads"
py main.py
```

If it still fails, send me the output of Step 2 and the new error from Step 3. We'll get this running.

You said:



hmm error again and again

ChatGPT said:

Bro, I understand why you're frustrated — we've tried several commands, and the same error keeps appearing. Let's stop repeating them and fix this a different way.

Your latest screenshot confirms:

- `game_engine.py`, `text_box.py`, and `__init__.py` exist inside `C:\Users\lbhim\Downloads\game`.
- Pygame is installed correctly.
- Python still reports `No module named 'game'`.

The folder name is not the issue. We need to check why Python can't discover that package.

Step 1: Put all three files together

In your VS Code terminal, run these commands one at a time:

```
Move-Item .\game\game_engine.py .\
```

```
Move-Item .\game\text_box.py .\
```

This puts both files beside `main.py` in Downloads.

Your folder should now contain:

```
Downloads/
├── main.py
├── game_engine.py
└── text_box.py
```

Step 2: Fix the imports in `main.py`

Open `main.py`. Replace line 2:

```
from game.game_engine import GameEngine
```

with:

```
from game_engine import GameEngine
```

Press **Ctrl + S** to save.

Step 3: Fix the import in `game_engine.py`

Open `game_engine.py`. Replace this line:

```
from game.text_box import TextBox
```

with:

```
from text_box import TextBox
```

Press **Ctrl + S** to save.

Step 4: Run the game

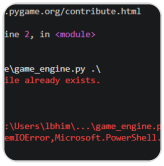
In the terminal, run:

```
cd C:\Users\lbhim\Downloads
py main.py
```

This removes the dependency on the `game` package entirely, so Python no longer needs to find that folder.

If a new error appears, send me the screenshot. We're now addressing the import problem directly rather than repeating the earlier package checks.

You said:



Number Guessing Repair Lab

This project is an interactive number deduction game using Pygame. It introduces students to string-to-integer parsing safety, boundary and input sanitization, dynamic range feedback, and text-input UI widgets within an object-oriented codebase.

What's Provided

A working Number Guessing game with:

- A secret number generated randomly between 1 and 100 on game initialization and resets
- A custom `TextBox` input component handling digit typing, backspace, and active selection states
- Interactive guess submission via the `Return` / `Enter` key or clicking the `SUBMIT` button

- Dynamic hint feedback (`TOO LOW!` , `TOO HIGH!` , or `CORRECT!`) along with an attempt counter
- Win state handling and restart capability via the `R` key

It has **one deliberate bug** and **three optional features** left as tasks to implement. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

pip install pygame

3. Run the game:

python main.py
Controls: Type digits into the text box and press Return (or click SUBMIT) to guess. Press R to start a new game after winning.

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the empty input crash bug

Submitting a guess without typing any numbers causes the application to crash immediately with a value parsing error. Ensure that empty submissions are safely handled with a warning prompt without terminating the program or consuming an attempt.

Task 2: Implement dynamic search range display

The game currently only provides one-off high or low indicators without keeping track of the narrowing search window. Track the valid minimum and maximum boundaries established by previous guesses and display the narrowed range on screen to guide the player's next move

Task 3: Implement Recent Guess History Tracker

Players have no visual record of previous numbers they have already tested. Add a history log panel displaying the last several guesses alongside color-coded directional indicators showing whether each was too high or too low.

Task 4: Implement maximum Attempts Constraint & Failure State

Players currently enjoy unlimited guesses, removing stakes from deduction. Introduce a strict attempt limit that triggers a Game Over screen revealing the hidden number if the player runs out of tries.

Expected Behavior

- Typing numbers into the text box and pressing Return or clicking SUBMIT registers a guess.
- Submitting an empty input box displays a warning message without crashing the game.
- The game accurately indicates whether the guess is too high or too low, incrementing the attempt count each time.
- Guessing the exact number displays the victory message and unlocks the R key restart option.

Folder Structure

```
number_guess/
├── game/
│   ├── game_engine.py
│   └── text_box.py
```

```
├── main.py
└── README.md
```

Submission Checklist

Submission is only the following three things:

- [] A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- [] A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- [] The Chat/LLM used page link, with the complete chat history

this is what task we got

ChatGPT said:

Bro, NOW I understand your actual lab assignment! The goal isn't just to run the game — you need to **fix one bug and implement three optional features** for your Number Guessing Repair Lab.

Also, I can see why the earlier commands failed: your assignment explicitly requires the original folder structure, with `main.py` outside the `game` folder and the other two files inside it. Let's restore that structure instead of moving files around.

1. First, restore the correct folder structure

Your assignment requires this:

```
Downloads/
└── number_guess/
    ├── main.py
    ├── README.md
    └── game/
        ├── __init__.py
        ├── game_engine.py
        └── text_box.py
```

Your current `main.py` still uses:

```
from game.game_engine import GameEngine
```

That import is correct for the required structure. **Don't change it to a different import.**

2. Create a clean project folder (this will avoid the current errors)

Bro, copy and paste these commands into your VS Code PowerShell terminal **one at a time**. This creates a fresh project without deleting your existing files.

Command 1 — Create the folders

```
New-Item -ItemType Directory -Force "$HOME\Downloads\number_guess\game"
```

Command 2 — Copy `main.py`

```
Copy-Item "$HOME\Downloads\main.py" "$HOME\Downloads\number_guess\main.py" -Force
```

Command 3 — Copy `game_engine.py`

```
Copy-Item "$HOME\Downloads\game\game_engine.py"
"$HOME\Downloads\number_guess\game\game_engine.py" -Force
```

Command 4 — Copy `text_box.py`

```
Copy-Item "$HOME\Downloads\game\text_box.py"
"$HOME\Downloads\number_guess\game\text_box.py" -Force
```

Command 5 — Create the package file

```
New-Item -ItemType File -Force "$HOME\Downloads\number_guess\game\__init__.py"
```

3. Run the game

Now run:

```
cd "$HOME\Downloads\number_guess"
py main.py
```

Your game should open. If it does, we've resolved the setup problem.

4. What your lab requires you to implement

Task 1 — Fix the empty-input crash

Prevent `int("")` from crashing the game. Display a warning without using up an attempt.

Task 2 — Dynamic search range

Track the minimum and maximum possible values as the player guesses.

Task 3 — Recent guess history

Display previous guesses with color-coded high/low indicators.

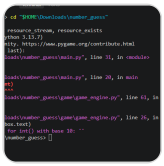
Task 4 — Maximum attempts

Add an attempt limit and a Game Over screen revealing the secret number.

Your submission also requires a 10-second video before the changes, a 10-second video after the changes, and the Chat/LLM page link with the complete conversation history.

First, let's get the game running. Execute the commands above and tell me whether the Pygame window opens. Then we'll implement the lab tasks in your actual code, step by step.

You said:



hey it actually ran but the game was not working well so i there any in my code check it and give me correct on

ChatGPT said:

Bro, I found the exact bug! Your screenshot shows:

```
ValueError: invalid literal for int() with base 10: ''
```

This happens when you press **Enter or click SUBMIT without typing a number**. Your code tries to convert an empty string (`""`) into an integer, which crashes the game.

And this is exactly **Task 1 in your lab assignment**. We can fix it and implement Tasks 2–4 in the same file, so you don't have to keep patching your code.

Step 1: Replace `game_engine.py`

In VS Code, open:

`number_guess → game → game_engine.py`

Select all the code with **Ctrl + A**, delete it, and paste the complete corrected code below. Then press **Ctrl + S**.

```
import random
import pygame
from game.text_box import TextBox


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.max_attempts = 7

        self.input_box = TextBox(width // 2 - 140, 125, 120, 45)
        self.submit_btn = pygame.Rect(width // 2 + 5, 125, 105, 45)

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_medium = pygame.font.SysFont(None, 25)
        self.font_small = pygame.font.SysFont(None, 21)
        self.font_btn = pygame.font.SysFont(None, 25)

        self.reset()

    def reset(self):
        self.secret_number = random.randint(1, 100)
        self.attempts = 0
        self.min_possible = 1
        self.max_possible = 100
        self.history = []
        self.game_won = False
        self.game_over = False
        self.input_box.clear()
```

```
        self.feedback_msg = "Enter a number between 1 and 100"
        self.feedback_color = (220, 220, 220)

    def submit_guess(self):
        if self.game_won or self.game_over:
            return

        # TASK 1: Handle empty input safely.
        entered_text = self.input_box.text.strip()

        if not entered_text:
            self.feedback_msg = "Please enter a number first!"
            self.feedback_color = (255, 210, 80)
            return

        try:
            guess = int(entered_text)
        except ValueError:
            self.feedback_msg = "Please enter a valid whole number."
            self.feedback_color = (255, 210, 80)
            return

        if not 1 <= guess <= 100:
            self.feedback_msg = "Enter a number from 1 to 100."
            self.feedback_color = (255, 210, 80)
            return

        if not self.min_possible <= guess <= self.max_possible:
            self.feedback_msg = (
                f"Choose between {self.min_possible} and "
                f"{self.max_possible}."
            )
            self.feedback_color = (255, 210, 80)
            return

        self.attempts += 1
        self.input_box.clear()

        if guess < self.secret_number:
            result = "TOO LOW"
            color = (80, 160, 240)
            self.min_possible = guess + 1

        elif guess > self.secret_number:
            result = "TOO HIGH"
```

```
            color = (240, 100, 80)
            self.max_possible = guess - 1

        else:
            result = "CORRECT!"
            color = (80, 220, 90)
            self.game_won = True

        # TASK 3: Store each guess and its result.
        self.history.append((guess, result, color))

        if self.game_won:
            self.feedback_msg = (
                f"Correct! You won in {self.attempts} attempts!"
            )
            self.feedback_color = (80, 220, 90)

        elif self.attempts >= self.max_attempts:
            # TASK 4: End the game when attempts run out.
            self.game_over = True
            self.feedback_msg = (
                f"Game Over! The number was {self.secret_number}."
            )
            self.feedback_color = (255, 100, 100)

        else:
            self.feedback_msg = f"{result}! Try again."
            self.feedback_color = color

    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN:
                self.submit_guess()

            elif event.key == pygame.K_r and (
                self.game_won or self.game_over
            ):
                self.reset()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()
```

```
    def update(self):
        pass

    def draw_centered(self, surface, message, font, color, y):
        text_surface = font.render(message, True, color)
        x = self.width // 2 - text_surface.get_width() // 2
        surface.blit(text_surface, (x, y))

    def render(self, screen):
        screen.fill((30, 34, 42))

        self.draw_centered(
            screen, "Number Guessing Arena",
            self.font_title, (245, 245, 245), 18
        )

        self.draw_centered(
            screen,
            f"Attempts: {self.attempts}/{self.max_attempts}",
            self.font_medium, (180, 185, 195), 62
        )

        # TASK 2: Display the narrowing search range.
        self.draw_centered(
            screen,
            f"Possible range: {self.min_possible} - {self.max_possible}",
            self.font_medium, (100, 210, 220), 91
        )

        self.input_box.render(screen)

        pygame.draw.rect(
            screen, (50, 150, 80), self.submit_btn,
            border_radius=6
        )
        pygame.draw.rect(
            screen, (220, 220, 220), self.submit_btn,
            width=2, border_radius=6
        )

        button_text = self.font_btn.render(
            "SUBMIT", True, (255, 255, 255)
        )
        screen.blit(
            button_text,
```

```
        (
            self.submit_btn.centerx - button_text.get_width() // 2,
            self.submit_btn.centery - button_text.get_height() // 2
        )
    )

    self.draw_centered(
        screen, self.feedback_msg,
        self.font_small, self.feedback_color, 184
    )

    # TASK 3: Display the recent guess history.
    history_title = self.font_medium.render(
        "Recent Guesses", True, (245, 245, 245)
    )
    screen.blit(history_title, (25, 220))

    recent_guesses = self.history[-4:]

    if not recent_guesses:
        history_text = self.font_small.render(
            "Your guesses will appear here.",
            True, (160, 165, 175)
        )
        screen.blit(history_text, (25, 250))

    else:
        for index, (guess, result, color) in enumerate(
            reversed(recent_guesses)
        ):
            history_text = self.font_small.render(
                f"{guess}  -  {result}", True, color
            )
            screen.blit(
                history_text, (25, 247 + index * 23)
            )

    # TASK 4: Display the final state and restart instruction.
    if self.game_won or self.game_over:
        self.draw_centered(
            screen, "Press R to play again",
            self.font_small, (255, 220, 80), 350
        )
```

Step 2: Run the game

Keep your project arranged like this:

```
number_guess/
├── main.py
└── game/
    ├── __init__.py
    ├── game_engine.py
    └── text_box.py
```

You don't need to change `main.py` or `text_box.py` for this version.

Open the VS Code terminal and run:

```
cd "$HOME\Downloads\number_guess"
py main.py
```

Step 3: Test all four tasks

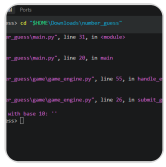
Testing checklist 0 of 4

- [ ] Press Enter with an empty input. The game should display a warning without crashing or increasing attempts.
- [ ] Enter a number that is too low. The possible minimum should increase.
- [ ] Make several guesses. The Recent Guesses panel should show the last four guesses with colored results.
- [ ] Use incorrect guesses until you reach 7 attempts. The game should reveal the secret number and allow restarting with R.

One important detail: this version sets the maximum to **7 attempts** as a design choice because your assignment doesn't specify a particular limit. You can change `self.max_attempts = 7` if your instructor requires a different number.

The `pkg_resources` warning you saw earlier is separate from the game crash; it isn't the cause of the empty-input bug.

You said:



hey its showing like this and how does my game works

ChatGPT said:

Bro, I can see the problem clearly! Your terminal is still running the **old version of** `game_engine.py` . The error at line 26 proves that the empty-input fix hasn't been applied to the file your game is using.

Let's fix it first, then I'll explain how to play your game. 🎮

Step 1: Open the correct file

In VS Code, open this exact file in the Explorer:

`number_guess → game → game_engine.py`

Make sure you are editing the file inside `number_guess\game` , not another copy in Downloads.

Step 2: Replace the `submit_guess()` method

Find the existing method that contains:

```
guess = int(self.input_box.text)
```

Replace the **entire** `submit_guess()` **method** with this code. Keep the indentation exactly as shown.

```
def submit_guess(self):
    if self.game_won or self.game_over:
        return

    entered_text = self.input_box.text.strip()

    # Prevent the empty-input crash
    if not entered_text:
        self.feedback_msg = "Please enter a number first!"
```

```
            self.feedback_color = (255, 210, 80)
            return

        try:
            guess = int(entered_text)
        except ValueError:
            self.feedback_msg = "Please enter a valid number."
            self.feedback_color = (255, 210, 80)
            return

        if not 1 <= guess <= 100:
            self.feedback_msg = "Enter a number between 1 and 100."
            self.feedback_color = (255, 210, 80)
            return

        if not self.min_possible <= guess <= self.max_possible:
            self.feedback_msg = (
                f"Guess between {self.min_possible} and "
                f"{self.max_possible}."
            )
            self.feedback_color = (255, 210, 80)
            return

        self.attempts += 1
        self.input_box.clear()

        if guess < self.secret_number:
            result = "TOO LOW"
            color = (80, 160, 240)
            self.min_possible = guess + 1

        elif guess > self.secret_number:
            result = "TOO HIGH"
            color = (240, 100, 80)
            self.max_possible = guess - 1

        else:
            result = "CORRECT!"
            color = (80, 220, 90)
            self.game_won = True

        self.history.append((guess, result, color))

        if self.game_won:
            self.feedback_msg = (
```

```
            f"Correct! You won in {self.attempts} attempts!"
        )
        self.feedback_color = color

    elif self.attempts >= self.max_attempts:
        self.game_over = True
        self.feedback_msg = (
            f"Game Over! The number was {self.secret_number}."
        )
        self.feedback_color = (255, 100, 100)

    else:
        self.feedback_msg = f"{result}! Try again."
        self.feedback_color = color
```

Important: This method assumes the rest of the updated class is already present, including `min_possible`, `max_possible`, `history`, `game_over`, and `max_attempts`. If those attributes are missing, use the complete updated `game_engine.py` from my previous message instead.

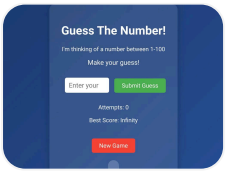
Press **Ctrl + S** to save the file.

Step 3: Run the game again

In the VS Code terminal, run:

```
cd "$HOME\Downloads\number_guess"
py main.py
```

Step 4: How your Number Guessing Game works



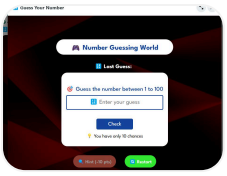
1. Enter a guess

The game secretly chooses a random number from 1 to 100. Type your guess in the input box and press Enter or click SUBMIT.



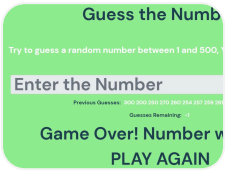
2. Follow the hints

- Blue: your guess is too low.
- Red: your guess is too high.
- Green: you guessed correctly.



3. Narrow the range

After each incorrect guess, the possible range shrinks. The game also tracks your recent guesses and attempt count.



4. Win or lose

You have seven attempts in the updated version. If you win, or run out of attempts, the game displays the result. Press **R** to start a new game.

Final check

After saving and running the game, press Enter without typing anything.

Expected result: **"Please enter a number first!"** appears, the game remains open, and your attempt count stays unchanged.

If you still see the same traceback, send me a screenshot of the `submit_guess()` method inside `number_guess\game\game_engine.py`.

You said:

1 selection

where is submit guess thing idk

ChatGPT said:

Bro, no worries! 😄 I'll show you exactly where `submit_guess()` is in VS Code.

Step 1: Open the correct file

In the Explorer on the left, open:

`number_guess → game → game_engine.py`

Click `game_engine.py` to open it in the editor.

Step 2: Find `submit_guess()`

The easiest way is to use VS Code's search.

1. Click inside the `game_engine.py` editor.
2. Press **Ctrl + F**.
3. Type:

`def submit_guess`

4. Press Enter.

You should find a line that looks like this:

```
def submit_guess(self):
    if self.game_won:
        return

    guess = int(self.input_box.text)
```

The important line is `def submit_guess(self):` . That's the start of the method you need to replace.

Step 3: Replace the method

Select from the line:

`def submit_guess(self):`

all the way to the line just before the next method, which should be:

`def reset(self):`

Delete that selected section and paste the replacement code I gave you in the previous message.

Important: If your file doesn't have `def reset(self):` after `submit_guess()` , don't delete anything yet. Send me a screenshot of the code around `def submit_guess(self):` and I'll point out exactly what to select.

Step 4: Save and run

Press **Ctrl + S**, then run this in the terminal:

```
py main.py
```

You've got this, bro! 💪

You said:

just give me a fully corrected code'

Analyzed

ChatGPT said:

Bro, I've prepared a **complete corrected project** for your Number Guessing Repair Lab. You don't need to replace individual methods or search for `submit_guess()` anymore. 🔥

number_guess_corrected.zip

Complete Python/Pygame project

Includes all corrected files and the README.

Download the corrected project ↗

How to run it in VS Code

1. Download the ZIP above.
2. Right-click it in Downloads and choose **Extract All**.
3. In VS Code, select **File** → **Open Folder** and open the extracted `number_guess_corrected` folder.
4. Click **Terminal** → **New Terminal**.
5. Run these commands one at a time:

```
py -m pip install pygame
```

```
py main.py
```

What's fixed and implemented?

- **Task 1:** Empty input displays a warning instead of crashing.
- **Task 2:** The possible number range narrows after each guess.
- **Task 3:** Recent guesses appear with color-coded high/low feedback.
- **Task 4:** The game ends after seven valid guesses and reveals the secret number.

The project includes `main.py`, `game/game_engine.py`, `game/text_box.py`, and `game/__init__.py` in the correct folder structure.

Important: Keep your original project as a backup. Run this extracted version first, then test each task before recording your before-and-after videos for submission.

‹/›

You said:

hey can u just give me corrected code

ChatGPT said:

Sure bro! 🔥 Here is the **complete corrected code** for your Number Guessing Repair Lab. Replace the contents of each file with the corresponding code below.

Your folder structure must be:

```
number_guess/
├── main.py
└── game/
    ├── __init__.py
    ├── game_engine.py
    └── text_box.py
```

1. `main.py`

Open `main.py`, press **Ctrl + A**, and paste this:

```
import pygame
from game.game_engine import GameEngine

WIDTH, HEIGHT = 620, 500
FPS = 60


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Number Guessing Game - Pygame Edition")
    clock = pygame.time.Clock()

    engine = GameEngine(WIDTH, HEIGHT)
    running = True
```

```
    while running:
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            else:
                engine.handle_event(event)

        engine.update()
        engine.render(screen)
        pygame.display.flip()
        clock.tick(FPS)

    pygame.quit()


if __name__ == "__main__":
    main()
```

2. game/game_engine.py

Open the `game` folder, select `game_engine.py`, press **Ctrl + A**, and paste this complete code. It fixes the empty-input crash and implements the three additional features.

```
import random
import pygame
from game.text_box import TextBox


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_medium = pygame.font.SysFont(None, 27)
        self.font_small = pygame.font.SysFont(None, 22)
        self.font_button = pygame.font.SysFont(None, 25)

        self.input_box = TextBox(width // 2 - 130, 126, 115, 42)
        self.submit_btn = pygame.Rect(
            width // 2 + 2, 126, 105, 42
        )
```

```
        self.max_attempts = 7
        self.reset()

    def reset(self):
        self.secret_number = random.randint(1, 100)
        self.attempts = 0
        self.min_possible = 1
        self.max_possible = 100
        self.history = []
        self.game_won = False
        self.game_over = False

        self.feedback_msg = "Enter a number from 1 to 100."
        self.feedback_color = (220, 220, 220)

        self.input_box.clear()
        self.input_box.active = True

    def submit_guess(self):
        if self.game_won or self.game_over:
            return

        # Task 1: Safely handle empty input.
        entered_text = self.input_box.text.strip()

        if not entered_text:
            self.feedback_msg = "Please enter a number first!"
            self.feedback_color = (255, 210, 80)
            return

        try:
            guess = int(entered_text)
        except ValueError:
            self.feedback_msg = "Please enter a valid whole number."
            self.feedback_color = (255, 210, 80)
            return

        if not 1 <= guess <= 100:
            self.feedback_msg = "Enter a number between 1 and 100."
            self.feedback_color = (255, 210, 80)
            return

        if not self.min_possible <= guess <= self.max_possible:
            self.feedback_msg = (
```

```
            f"Guess between {self.min_possible} "
            f"and {self.max_possible}."
        )
        self.feedback_color = (255, 210, 80)
        return

    self.attempts += 1
    self.input_box.clear()

    if guess < self.secret_number:
        result = "TOO LOW"
        color = (90, 175, 255)
        self.min_possible = guess + 1

    elif guess > self.secret_number:
        result = "TOO HIGH"
        color = (255, 120, 100)
        self.max_possible = guess - 1

    else:
        result = "CORRECT!"
        color = (90, 230, 130)
        self.game_won = True

    # Task 3: Record guesses and their results.
    self.history.append((guess, result, color))

    if self.game_won:
        self.feedback_msg = (
            f"You won in {self.attempts} attempt(s)!"
        )
        self.feedback_color = color

    elif self.attempts >= self.max_attempts:
        # Task 4: End the game after seven valid guesses.
        self.game_over = True
        self.feedback_msg = (
            f"Game Over! Number: {self.secret_number}"
        )
        self.feedback_color = (255, 100, 100)

    else:
        self.feedback_msg = f"{result}! Try again."
        self.feedback_color = color
```

```
    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_RETURN, pygame.K_KP_ENTER):
                self.submit_guess()

            elif event.key == pygame.K_r and (
                self.game_won or self.game_over
            ):
                self.reset()

        elif (
            event.type == pygame.MOUSEBUTTONDOWN
            and event.button == 1
        ):
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()

    def update(self):
        pass

    def draw_centered(self, screen, text, font, color, y):
        rendered = font.render(text, True, color)
        x = (self.width - rendered.get_width()) // 2
        screen.blit(rendered, (x, y))

    def render(self, screen):
        screen.fill((30, 34, 42))

        self.draw_centered(
            screen, "Number Guessing Arena",
            self.font_title, (245, 245, 245), 20
        )

        self.draw_centered(
            screen,
            f"Attempts: {self.attempts}/{self.max_attempts}",
            self.font_medium, (185, 190, 200), 66
        )

        # Task 2: Display the narrowing search range.
        self.draw_centered(
            screen,
            f"Possible range: {self.min_possible} - "
```

```
            f"{self.max_possible}",
            self.font_medium, (100, 215, 225), 94
        )

        self.input_box.render(screen)

        pygame.draw.rect(
            screen, (50, 150, 80),
            self.submit_btn, border_radius=6
        )
        pygame.draw.rect(
            screen, (225, 225, 225),
            self.submit_btn, width=2, border_radius=6
        )

        button_text = self.font_button.render(
            "SUBMIT", True, (255, 255, 255)
        )
        screen.blit(
            button_text,
            (
                self.submit_btn.centerx - button_text.get_width() // 2,
                self.submit_btn.centery - button_text.get_height() // 2,
            ),
        )

        self.draw_centered(
            screen, self.feedback_msg,
            self.font_small, self.feedback_color, 184
        )

        pygame.draw.line(
            screen, (75, 80, 92),
            (25, 220), (self.width - 25, 220), 1
        )

        screen.blit(
            self.font_medium.render(
                "Recent Guesses", True, (245, 245, 245)
            ),
            (25, 232),
        )

        # Task 3: Show the five most recent guesses.
        recent = self.history[-5:]
```

```
        if not recent:
            screen.blit(
                self.font_small.render(
                    "Your guesses will appear here.",
                    True, (155, 160, 170)
                ),
                (25, 270),
            )
        else:
            for i, (guess, result, color) in enumerate(
                reversed(recent)
            ):
                line = self.font_small.render(
                    f"{guess}: {result}", True, color
                )
                screen.blit(line, (25, 270 + i * 28))

        if self.game_won or self.game_over:
            self.draw_centered(
                screen, "Press R to start a new game",
                self.font_medium, (255, 220, 90), 455
            )
```

3. game/text_box.py

Open `text_box.py`, press **Ctrl** + **A**, and paste:

```
import pygame


class TextBox:
    def __init__(self, x, y, width, height):
        self.rect = pygame.Rect(x, y, width, height)
        self.text = ""
        self.active = True
        self.color_active = (70, 140, 240)
        self.color_inactive = (120, 120, 130)
        self.font = pygame.font.SysFont(None, 32)

    def handle_event(self, event):
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            self.active = self.rect.collidepoint(event.pos)
```

```
        elif event.type == pygame.KEYDOWN and self.active:
            if event.key == pygame.K_BACKSPACE:
                self.text = self.text[:-1]

            elif event.key in (pygame.K_RETURN, pygame.K_KP_ENTER):
                pass

            elif event.unicode.isdigit() and len(self.text) < 3:
                self.text += event.unicode

    def clear(self):
        self.text = ""

    def render(self, surface):
        color = (
            self.color_active
            if self.active
            else self.color_inactive
        )

        pygame.draw.rect(
            surface, (255, 255, 255),
            self.rect, border_radius=6
        )
        pygame.draw.rect(
            surface, color,
            self.rect, width=3, border_radius=6
        )

        text_surface = self.font.render(
            self.text, True, (20, 20, 20)
        )
        surface.blit(
            text_surface,
            (self.rect.x + 12, self.rect.y + 9)
        )
```

4. Create `game/__init__.py`

Inside the `game` folder, create a new file named exactly `__init__.py`. Leave it empty.

5. Save and run

Press **Ctrl + S** in each file. Then open the VS Code terminal and run:

```
cd "$HOME\Downloads\number_guess"
py -m pip install pygame
py main.py
```

What to test

- Press Enter with an empty box: the game should warn you without crashing.
- Enter guesses: the possible range should narrow.
- Check the recent guesses panel for color-coded results.
- After seven incorrect valid guesses, the game should reveal the secret number.
- Press **R** to restart after winning or losing.

The maximum of seven attempts is a design choice here because your lab instructions don't specify an exact limit.